

Roteiro de Estudos

Avaliação Parcial 1 (AVP1)

Disciplina: Frameworks **Professor:** Ermeson Silva

1. Panorama da prova

A AVP1 cobre a primeira parte da ementa da disciplina:

- Introdução a linguagens de script para a web (JavaScript)
 - Processamento do lado do cliente
 - Variáveis, constantes, vetores, objetos, desvios condicionais e estruturas de repetição
 - Modularização com o uso de funções
 - Manipulação de elementos (DOM)
-

2. Aula 02 — Introdução ao JavaScript

2.1 Fundamentos

- JavaScript é interpretado pelo navegador (não precisa de compilador).
- É **case sensitive** (**Fruta** ≠ **fruta**).
- Tipagem **dinâmica e fraca**: o tipo da variável é definido em tempo de execução e pode mudar.
- Roda no cliente (navegador) ou no servidor (Node.js).
- Segue a especificação **ECMAScript (ECMA 262)**.

2.2 Inserindo JavaScript no HTML

```
<script>  
  // código JS aqui
```

```
</script>
```

Ou em arquivo externo:

```
<script src="programa.js"></script>
```

2.3 Variáveis: `var`, `let`, `const`

Palavra-chave	Escopo	Redeclaração	Hoisting
<code>var</code>	função ou global	permitida	sim (valor <code>undefined</code>)
<code>let</code>	bloco { }	não permitida	sim, mas em "zona morta" (gera erro se usada antes da declaração)
<code>const</code>	bloco { }	não permitida	mesmo comportamento de <code>let</code> ; valor não pode ser reatribuído

Pegadinha clássica de prova: diferença entre escopo de função (`var`) e escopo de bloco (`let/const`), e o conceito de *hoisting*.

2.4 Tipos de dados

- `String`, `Number`, `Boolean`, `Array`, `Object`, `Function`
- Exemplo de tipagem dinâmica:

```
var x;           // undefined
x = 5;           // number
x = "John";      // string
x = true;        // boolean
```

2.5 Operadores

- **Aritméticos:** `** * / + - %`
- **Relacionais:** `== != === !== < > <= >=`
 - `==` compara valor; `===` compara valor e tipo (idêntico).
- **Lógicos:** `&&` (E) `||` (OU) `!` (negação) `!!` (dupla negação)

2.6 Estruturas condicionais

```
if (teste_logico) {  
    // se verdadeiro  
} else {  
    // se falso  
}
```

Também: `switch/case` (ver exemplo do "dia da semana" na aula).

2.7 Primeira manipulação do DOM

```
document.getElementById("paragrafo").innerHTML =  
"Hello!";
```

2.8 Exercícios que caíram em aula (revisar de cabeça)

- Calcular a média de 3 notas e dizer se o aluno foi aprovado (média ≥ 6).
- Ler um número de 1 a 7 e retornar o dia da semana (`switch`).
- Verificar se um número é par ou ímpar (`% 2 === 0`).
- Calcular o IMC (`peso / altura2`) e classificar a faixa.

3. Aula 03 — Loops, Arrays e Funções

3.1 Estruturas de repetição

```
// for – quando se sabe o número de repetições
for (var i = 0; i < 5; i++) { }

// while – repete enquanto a condição for verdadeira
let contador = 0;
while (contador <= 20) {
  contador++;
}
```

- **Exercício clássico:** imprimir os números pares de 0 a 100, usando `for` e usando `while`.

3.2 Arrays

- Estrutura dinâmica que armazena coleções de valores de **qualquer tipo** (inclusive misturados).
- Índices começam em **0**.

```
let cores = ["vermelho", "azul", "verde"];
cores[1] = "manga";           // modifica um elemento
let pessoas = [
  { nome: "Alice", idade: 25 },
  { nome: "Bob", idade: 30 },
];
```

3.3 Métodos de array (saber o que cada um faz e se ele **muda o array original** ou **retorna um novo**)

Método	O que faz	Muda o array original?
<code>push()</code>	adiciona no final	sim
<code>pop()</code>	remove do final	sim

Método	O que faz	Muda o array original?
<code>unshift()</code>	adiciona no início	sim
<code>shift()</code>	remove do início	sim
<code>map()</code>	transforma cada elemento	não (retorna novo array)
<code>filter()</code>	filtra elementos por condição	não (retorna novo array)
<code>reduce()</code>	reduz o array a um único valor	não
<code>find()</code>	retorna o primeiro elemento que satisfaz a condição	não

3.4 Percorrendo arrays

```
for (let i = 0; i < frutas.length; i++) {  
  console.log(frutas[i]); }  
  
let i = 0;  
while (i < frutas.length) { console.log(frutas[i]);  
  i++; }  
  
frutas.forEach(function (fruta) {  
  console.log(fruta); });
```

Pegadinha: `forEach` não tem `break` — não dá para parar a iteração no meio.

3.5 Funções

```
function media_final(nota1, nota2, nota3) {  
  return (nota1 + nota2 + nota3) / 3;
```

```
}
```

- Parâmetros levam valores de "fora" para "dentro" da função.
- Uma função só executa o que está dentro dela quando é **chamada**.

3.6 Exercícios que caíram em aula (revisar de cabeça)

- Função que diz se um número é par ou ímpar.
- Função que recebe 3 notas, calcula a média e diz se está aprovado (média ≥ 7).
- Somar apenas os números pares de um array, usando **for**.
- Usar **forEach** para imprimir o nome de cada objeto de um array.
- Combinar **map** + **filter** (ex.: colocar strings em maiúsculas e depois filtrar as que começam com uma letra).
- Ler dois números de inputs HTML e multiplicar via JS (integração DOM + função).

4. Aula 04 — Manipulação do DOM

4.1 O que é o DOM

- **DOM (Document Object Model)**: representação da página HTML como uma **árvore de objetos/nós**.
- Tipos de nó: elemento, atributo, texto, comentário, e o nó **document** (raiz).
- Relações: nó **pai**, nós **filhos**, nós **irmãos** (mesmo pai), nó **folha** (sem filhos), **root** (topo, não tem pai).

4.2 Selecionando elementos

```
document.getElementById('meuId');           // um
elemento, por id
document.querySelector('.minhaClasse');     //
primeiro elemento que casa com o seletor CSS
```

```
document.querySelectorAll('div'); //  
todos os elementos (NodeList)
```

4.3 Alterando conteúdo, atributos e estilo

```
element.innerHTML = '<strong>Novo  
conteúdo</strong>'; // interpreta HTML  
element.textContent = 'Apenas texto';  
// texto puro  
  
img.setAttribute('src', 'novaImagem.jpg');  
  
element.style.color = 'blue';  
element.style.fontSize = '20px';
```

Pegadinha: diferença entre `innerHTML` (interpreta tags) e `textContent` (trata tudo como texto).

4.4 Trabalhando com classes CSS via JS

```
element.classList.add('novaClasse');  
element.classList.remove('antigaClasse');  
element.classList.toggle('red'); // alterna:  
adiciona se não tem, remove se tem
```

4.5 Criando e removendo elementos

```
const novo = document.createElement('div');  
novo.textContent = 'Eu sou um novo elemento';  
document.body.appendChild(novo);  
  
element.remove(); // remove da árvore DOM
```

4.6 Eventos

```
button.addEventListener('click', function () {  
  alert('Botão clicado!');  
});
```

- Eventos comuns: `click`, `mouseover`, `keydown`, `submit`, `scroll`.

4.7 Desafios que caíram em aula (revisar de cabeça)

- **"Bagunça na Tela"**: botão que cria elementos (`createElement` + `appendChild`) em posição e cor aleatórias, e botão que os remove todos (`querySelectorAll` + `remove`).
- **Lista dinâmica**: input + botão que adiciona itens a uma `<ul>` (`createElement('li')` + `appendChild`).
- **Banner interativo (troca de conteúdo por clique)**: capturar vários elementos com `querySelector`, usar `addEventListener` com *arrow function*, trocar `src`, `innerHTML` e classe `active` conforme o botão clicado — inclui uma função para "limpar" o estado anterior antes de aplicar o novo (`classList.remove` em loop com `forEach`).

5. Checklist — o que você precisa saber fazer sem consultar

- ☐ Declarar variáveis com `let/const` e explicar por que evitar `var`
- ☐ Explicar a diferença entre `==` e `===`
- ☐ Escrever um `if/else` e um `switch/case`
- ☐ Escrever um `for` e um `while` equivalentes
- ☐ Criar, acessar, modificar e percorrer um array
- ☐ Usar `push`, `pop`, `map`, `filter`, `find` corretamente
- ☐ Escrever uma função com parâmetros e `return`
- ☐ Selecionar elementos com `getElementById` / `querySelector` / `querySelectorAll`

- ☐ Alterar conteúdo (`innerHTML/textContent`), atributo (`setAttribute`) e estilo (`.style`) de um elemento
 - ☐ Adicionar/remover classes com `classList`
 - ☐ Criar e remover elementos dinamicamente (`createElement`, `appendChild`, `remove`)
 - ☐ Registrar um evento com `addEventListener`
 - ☐ Integrar um formulário HTML simples com uma função JS (ler `input.value`, calcular, exibir resultado)
-

6. Sugestão de cronograma (4 dias antes da prova)

Dia	Foco
D-4	Rer Aula 02 (fundamentos, variáveis, operadores, condicionais) e refazer os 4 exercícios da aula sem olhar a solução
D-3	Rer Aula 03 (loops, arrays, métodos, funções) e refazer os exercícios de arrays e funções
D-2	Rer Aula 04 (DOM, seleção, manipulação, eventos) e refazer o desafio "Bagunça na Tela" do zero
D-1	Revisar a checklist da seção 5, recriar de memória o desafio do banner interativo (aula 04) e refazer os desafios de par/ímpar, média e arrays da aula 02/03

7. Pontos de atenção (erros comuns em prova)

1. Confundir escopo de `var` (função) com escopo de `let/const` (bloco).
2. Esquecer que índices de array começam em **0**.
3. Usar `==` quando a questão pede comparação estrita (`===`).
4. Confundir `innerHTML` (aceita HTML) com `textContent` (só texto).
5. Achar que `map/filter` alteram o array original — eles **retornam um novo array**.

6. Esquecer o **return** dentro de uma função (a função "não devolve nada" sem ele).
 7. Selecionar um elemento antes de ele existir no HTML (o script deve rodar depois do elemento estar na página, ou dentro de um listener).
-

Fontes de estudo

- Materiais de aula (disponíveis no AVA e no repositório da disciplina);
 - Códigos construídos em sala de aula;
 - Documentação W3C: <https://www.w3schools.com>
-

Atenção: Este guia é apenas um resumo e não substitui o estudo dos conteúdos e códigos em sua totalidade.

Atenção 2: Este guia foi gerado por um agente de IA. Ele pode cometer erros. No entanto, os tópicos a serem estudados estão corretos.

